

千葉工業大学様
AI 教育（GPGPU）演習用計算機システム
利用者向け操作マニュアル



People and Computing Power

0. 本システムについて

1. Quick Start Guide

2. RootlessDockerの利用方法

2-1 docker pull

2-2 docker run

2-3 docker build

2-4 docker push

2-5 rootless dockerの注意事項

2-6 dockerコマンド概要図

2-7 docker hub注意事項

3. Harborの利用方法

3-1 ウェブブラウザへのアクセス

3-2 利用可能なプロジェクト一覧

3-3 Harborへコンテナをpull/pushする

4. Slurmの利用方法

4-1 ジョブの投入

4-2 dockerコンテナジョブの投入

4-3 投入中のジョブの確認/キャンセル

4-4 ジョブ終了後の確認

4-5 本システムで非推奨のジョブ

4-6 演習モードへ移行した際の挙動

5. GPGPU計算機の利用にあたって

5.1 ストレージ容量の制限

5.2 Slurmのリソース制限(class)

5.3 Slurmのリソース制限(research)

5.4

6. 開発事例

7. 教員向け情報

7-1 共有データ置き場について

7-2 教員ユーザの権限について

7-3 授業での利用にあたって

アカウントについて

本システムはMARINE アカウントでログオンして利用できます。
ただし以下の制限がありますのでご注意ください。

- sshでのアクセス、レジストリサーバ(Harbor)へのアクセスの際のアカウントは**大文字小文字が区別されます**。
発行されたアカウント通り入力ください。
- レジストリサーバ(Harbor)の各ユーザのプライベートプロジェクト名は**MARINE アカウントの大文字を小文字化したもの**です。
- **MARINE アカウントの発行翌日から利用可能**です。
- 学生の転学科等があった場合、翌日以降に反映されます。
- 演習モードに設定されている場合、学生は指定された学科の学生以外はログオンできません。

本マニュアルの記載ルールに関して

本マニュアルでコマンドを記載するとき、
どのサーバーで実行するのかを下記のように明記しています。

一般的なコマンドの説明であり、実行端末を特定しない場合(\$のみ):

```
$ docker run -it --rm centos:8/bin/bash
```

BYOD等、本システム以外の端末上で実行することを想定している場合:

```
(other)$ docker build -t pytorch_test:1.0 .
```

本システム(GPUサーバー)上で実行することを想定している場合:

```
(gpumng)$ qsub testjob.sh
```

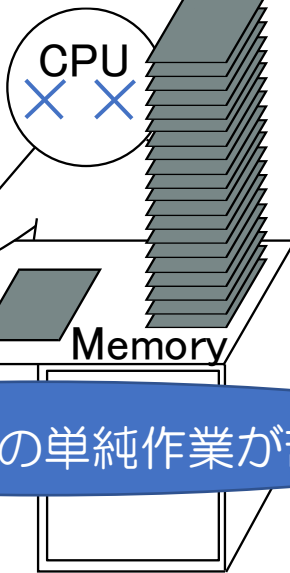
0. 本システムについて

CPUとGPU

現在のコンピュータの主流な構成

CPU(頭脳)

演算処理を行いデータ処理・機器の制御を行う装置



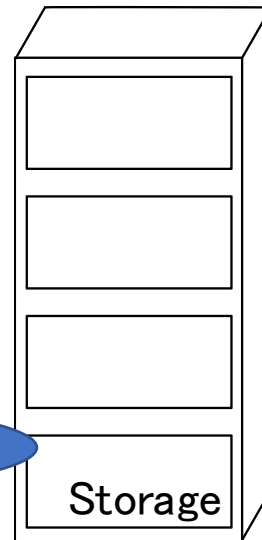
CPUは大量の単純作業が苦手

Memory(勉強机)

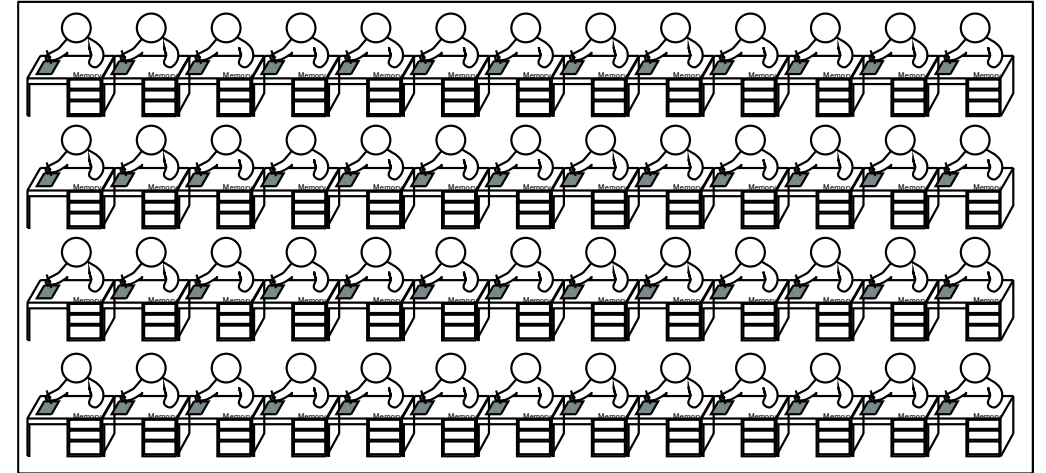
CPUから容易にアクセスできるデータの一時置き場

Storage(本棚)

データを保管しておくための装置



GPU



GPU(頭脳)

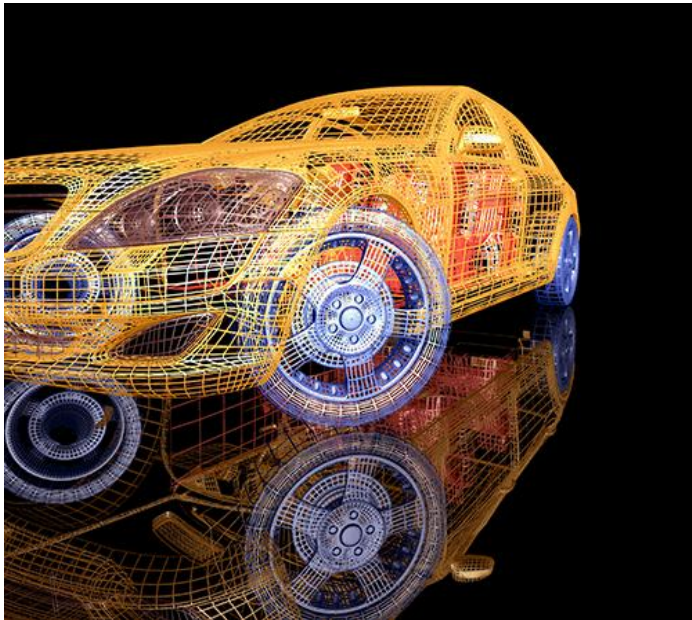
数の多い単純作業に特化した演算装置

複雑な作業ができるCPUと、数の多い単純作業に特化したGPUをどちらも載せ、用途に応じて使い分ける

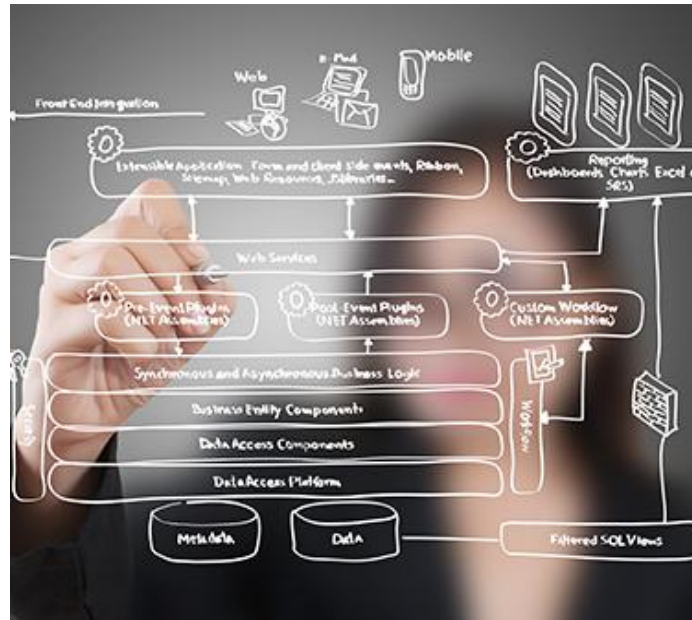
GPUが得意な計算

Graphics Processing Unit (GPU) で何ができるか？

画像処理



(一部の)科学計算



深層学習



疑似量子計算
時間発展方程式
ランダムサンプリング
行列計算
...

今後サーバーを利用するうえで必要になる知識

世界のスパコンランキングTOP10(2024年11月現在)
富岳以外は全てCPU/GPU混載システム

Rank	System
1	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A , Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States
2	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X , Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States
3	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max , Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States
4	Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100 , NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States
5	HPC6 - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X , Slingshot-11, RHEL 8.9, HPE Eni S.p.A. Italy
6	Supercomputer Fugaku - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu RIKEN Center for Computational Science Japan
7	Alps - HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip , Slingshot-11, HPE Cray OS, HPE Swiss National Supercomputing Centre (CSCS) Switzerland
8	LUMI - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X , Slingshot-11, HPE EuroHPC/CSC Finland
9	Leonardo - Bull/Sequana YH2000, Xeon Platinum 8358 32C 2.6GHz, NVIDIA A100-SXM4 64 GB , Quad-rail NVIDIA HDR100 Infiniband, EVIDEN EuroHPC/CINECA Italy
10	Tuolumne - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A , Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States

- CPU/GPU混載システムの主流化

⇒ CPU・GPUの特性を理解し、問題に応じて使い分けられること

- HWの多様化に伴い、環境設定が複雑化

⇒ 個別の環境設定をパッケージングできるコンテナ環境や仮想化技術を扱えること

- 高性能クラスターシステムの需要増

⇒ ジョブスケジューラ等、共同利用環境の利用方法を身につけること

システム概要(津田沼・新習志野キャンパス共通)

sshでログイン

sbatchでジョブをスケジューラに投入

新習志野

津田沼

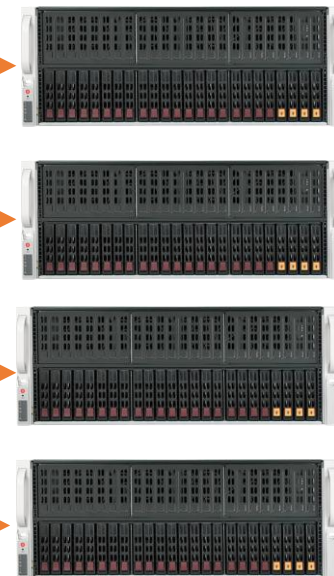
ログイン兼ファイルサーバー
gpumng.cle.it-chiba.ac.jp
gpumng2.cle.it-chiba.ac.jp

ssh

sbatch



GPGPU計算サーバー



演習室端末



BYOD端末



動作が軽量なコンテナ型仮想環境



コンテナの保管・管理・サーバー間移動



計算資源を各ジョブに分配

リソースが空いたサーバーでジョブが実行

*空きリソースがない場合、ジョブは待機状態



仕様

OS	・ Ubuntu 24.04 LTS
CPU	・ Xeon Gold 6426Y(16Core,2.5GHz,185W) x2
Memory	・ 1TB(64GB DDR5-4800 ECC RIMM x16)
Storage	・ 960GB SATA SSD x2 (System, Backup) ・ 3.85TB 2.5" NVMe x4
GPU	・ NVIDIA RTX A4500 (20GB,200W) x10

台数：4台

新習志野キャンパスのシステムは、合計40枚のA4500を搭載しています。
演習時間中など、短時間で多人数が集中して利用することを想定しています。
* 演習時間以外でも利用可能です。



仕様

OS	・ Ubuntu 24.04 LTS
CPU	・ Xeon Gold 6426Y(16Core,2.5GHz,185W) x2
Memory	・ 1TB(64GB DDR5-4800 ECC RIMM x16)
Storage	・ 960GB SATA SSD x2 (System, Backup) ・ 3.85TB 2.5" SATA SSD x4
GPU	・ NVIDIA H100 (94GB,400W) x8

台数：1台

津田沼キャンパスのシステムは、ハイエンドGPU H100 NVLを合計8枚搭載しています。
メモリは1GPUあたり94GB搭載しており、A4500(20GB)ではメモリに載りきらないような
計算負荷の高いワークロードへの利用を想定しています。

GPU性能比較

新習志野



津田沼



	A4500	H100 NVL
GPUアーキテクチャ	Ampere	Hopper
ベースクロック	1065MHz	1095MHz
GPUメモリ	20GB	94GB
メモリ帯域幅	640GB/s	3917GB/s
CUDA コア数	7168	16896
Tensorコア数	224	528
FP16	23.7TFLOPS	134TFLOPS
FP16 (Tensor Core)	95TFLOPS	989TFLOPS
FP32	23.7TFLOPS	66.9TFLOPS
FP32 (Tensor Core)	47.3TFLOPS	495TFLOPS
FP64	0.370TFLOPS	33.5TFLOPS

想定ワークフロー

- ①dockerコンテナと、コンテナ上で動くプログラムを作成
- ②Harborへdockerコンテナをpush/その他のファイルをgpumngにscpで移動
- ③gpumngサーバにログインして、Harborからコンテナをpull
- ④slurmにジョブを投入するためのスクリプトを作成
- ⑤sbatchでジョブを投入
- ⑥ジョブ終了後、scpで結果をローカル環境に移動
- ⑦解析

gpumng/gpumng2で実行

1. Quick Start Guide

1-1 本システムへのログイン

下記コマンドで各システムのログイン兼ファイルサーバでへログインして下さい。

新習志野

```
(other)$ ssh <ユーザー名>@gpumng.cle.it-chiba.ac.jp
```

津田沼

```
(other)$ ssh <ユーザー名>@gpumng2.cle.it-chiba.ac.jp
```

初回ログイン時は、下記2点に注意してください。

- 初回ログイン時は、初期設定が行われます。
初期設定中は強制ログアウトなどを実行しないでください。
初期設定中に強制終了などを実行し、適切に処理されなかった場合、
現在のアカウントを削除して再作成する必要があります。
- 下記コマンドを実行し、レジストリサーバ(Harbor)へログインしてください
(ユーザー名・パスワードは本システムへログインした際のものと同一です)

新習志野

```
(gpumng)$ docker login gpumng.cle.it-chiba.ac.jp
```

津田沼

```
(gpumng)$ docker login gpumng2.cle.it-chiba.ac.jp
```

1-2 ジョブの実行

本システムを利用するためのサンプルコマンドをdemoディレクトリに用意しています

```
(gpumng)$ cd demo/01_first_launch  
(gpumng)$ sbatch test_lec1.job
```

test_lec1.job

test_lec1.jobファイルの内容で
slurmにジョブを投入

・sbatchでジョブ投入後、

```
(gpumng)$ squeue
```

でジョブの実行状態を確認してください

```
hpc@gpumng:~/demos/demo_1/01_first_launch$ squeue  
      JOBID PARTITION    NAME    USER  ST       TIME  NODES NODELIST(REASON)  
      371      class test_lec    hpc   R          0:01      1 gpu01
```

・出力結果はslurm-XXX.outで確認できます

```
#!/bin/bash  
  
#SBATCH --partition=class  
##### class または research を設定して下さい。各パーティションの  
#####   class: CPU 3コア / GPU 1枚 / メモリ 90GB / 最大実行時間  
#####   research: CPU 32コア / GPU 10枚 / メモリ 960GB / 最大実行時間  
  
#SBATCH --nodes=1  
##### 計算時に利用する計算ノード数です。本システムでは複数ノード  
##### での計算はサポートしていません。  
  
#SBATCH --tasks-per-node=1  
##### 計算時に利用するタスク数です。本システムでは1に設定して  
##### います。  
  
#SBATCH --cpus-per-task=1  
##### 計算時に利用するCPUコア数です。選択したパーティションの  
##### リソースは1コアです。  
  
#SBATCH --gpus-per-task=1  
##### 計算時に利用するGPU枚数です。選択したパーティションの  
##### リソースは1枚です。  
  
#SBATCH --mem=90GB  
##### 計算時に利用するメモリ容量です。選択したパーティションの  
##### リソースは90GBです。  
  
##### 以下、実行したいコマンドを記入してください  
echo "GPUの認識枚数:"  
nvidia-smi  
echo ""  
  
echo "カレントディレクトリの確認:"  
pwd  
echo ""  
  
echo "pythonからプログラムの実行"  
python3 test.py  
echo ""  
  
echo "30秒待機"  
sleep 30
```

1-3 コンテナジョブの実行

本システムを利用するためのサンプルコマンドをdemoディレクトリに用意しています

```
(gpumng)$ cd demo/03_using_container_on_slurm  
(gpumng)$ sbatch test_lec3.job
```

test_lec3.jobファイルの内容で
slurmにジョブを投入

初回実行時、almalinux:9のダウンロードを実行します。

下記コマンドで現在取得しているコンテナイメージ一覧を
確認できます。

```
(gpumng)$ docker images
```

```
#!/bin/bash  
#SBATCH --partition=class  
#SBATCH --nodes=1  
#SBATCH --tasks-per-node=1  
#SBATCH --cpus-per-task=3  
#SBATCH --gpus-per-task=1  
#SBATCH --mem=90G  
  
##### コンテナの使用  
#SBATCH --container=almalinux:9  
  
echo "OS情報を表示"      使用するコンテナを記述  
cat /etc/os-release  
echo ""  
  
echo "カレントディレクトリの表示"  
pwd  
echo ""  
  
echo "カレントディレクトリ内を一覧表示"  
ls -la  
echo ""  
  
echo "共有データ領域の中身を一覧表示"  
ls -la /data  
echo ""  
  
##### 共有データのマウント先  
echo "実行ユーザーの確認"  
whoami  
echo ""  
  
echo "コンテナ内でGPUが見えるか確認"  
nvidia-smi  
echo ""  
  
#カレントディレクトリにファイルを作成  
echo "Hello from container" > output.dat
```

▶ 2. Rootless Dockerの利用方法



2. Docker利用方法

Dockerは、非常に動作が軽量なコンテナ型仮想環境であり、様々な用途で広く活用されています。
dockerには下記のようなメリットがあります。

- ①アプリケーション毎に異なるコンテナを作成することで、設定の競合などを防ぐ
- ②docker hub等から既存のdockerイメージをpullすることで、
アプリ実行までに必要な設定作業を省く
- ③管理者権限がない環境であってもコンテナ内であれば管理者権限を行使できる

本章では、dockerの一般的な使い方に関して説明します。



- 2-1 docker pull
- 2-2 docker run
- 2-3 docker build
- 2-4 docker push

2-1 docker pull

docker hubやNGC等、web上に様々なdockerイメージが公開されており、これらの設定済みコンテナイメージをpullしてそのまま使うことができます

▼Docker Hub

<https://hub.docker.com/>

▼NGC

<https://catalog.ngc.nvidia.com/>

例1) docker hubから

イメージ名(リポジトリ名): centos タグ名: 8
というdockerイメージをpullする

```
$ docker pull centos:8
```

例2) NGCから

イメージ(リポジトリ)名: nvidia/pytorch タグ名: 23.09-py3
というdockerイメージをpullする

```
$ docker pull nvcr.io/nvidia/pytorch:23.09-py3
```

2-2 docker run

dockerイメージをpullしたら、下記コマンドを実行することでコンテナが作成され、ログインできます。

```
$ docker run -it --rm centos:8 /bin/bash
```

① ②

- ①インタラクティブモードでcentos:8にログイン
- ②実行終了後、自動でコンテナを削除

ログインせずに指定のプログラムを実行することもできます。

```
$ docker run --rm centos:8 cat /etc/os-release
```

①

- ①実行したいコマンドを記述

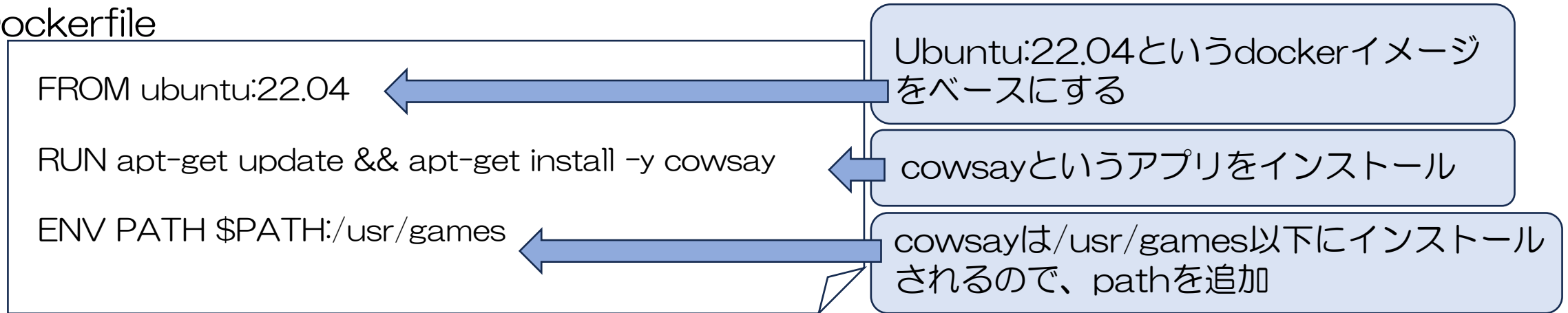
補足) イメージとコンテナの違い

- イメージ：コンテナを作成するための設計図（docker imagesで一覧表示）
- コンテナ：イメージから作成された実行環境（docker ps -aで一覧表示）
 - *上の例でrmオプションをつけなかった場合、実行終了後もコンテナは削除されずに残ります。

2-3 docker build

既存のdockerイメージをベースにして、新しいdockerイメージを作成(ビルド)することもできます。ビルドのためにはDockerfileという名前のファイルを作成し、設定内容を記述していきます。

Dockerfile



Dockerfileが置いてあるフォルダで下記を実行し、dockerイメージubuntu_cowsay:1.0を作成

```
$ docker build -t ubuntu_cowsay:1.0 .
```

作成したdockerイメージからコンテナを実行

```
$ docker run --rm ubuntu_cowsay:1.0 cowsay "Hello World"
```



2-4 docker push

新規作成したdockerイメージをHarborレジストリサーバに登録(push)することで、

- データの差分によるバージョン管理
- 他システムで作成したコンテナイメージをpull

することができるようになります。

① コンテナイメージの名前を変更

新習志野

```
$ docker tag ubuntu_cowsay:1.0 gpumng.cle.it-chiba.ac.jp/<username>/testimage-hub:1.0
```

津田沼

```
$ docker tag ubuntu_cowsay:1.0 gpumng2.cle.it-chiba.ac.jp/<username>/testimage-hub:1.0
```

pushする際の命名ルールは一般的に下記に基づきます。

<pushする先のサーバー名>/<プロジェクト名>/<リポジトリ名>:<タグ名>

*プロジェクト名に関しては、次章で詳述します。

② Docker pushコマンドでdocker hubに登録

新習志野

```
$ docker push gpumng.cle.it-chiba.ac.jp/<username>/testimage-hub:1.0
```

津田沼

```
$ docker push gpumng2.cle.it-chiba.ac.jp/<username>/testimage-hub:1.0
```

2-5 rootless dockerの注意事項

本システムの管理サーバ(gpumng)にはrootlessモードのdockerを採用しております。
通常の管理者権限が必要なrootfulなdockerとは以下の点で異なりますのでご注意ください。

- 同一サーバー内の別のユーザーとコンテナイメージを共有しません。
- コンテナ内ではrootユーザとして振舞います。
ただし、rootfulなdockerとは異なりシステム上のrootユーザーではなく、
コンテナを作成した一般ユーザーをrootとして疑似的にマッピングしたものになります
- コンテナ内でrootユーザが作成したファイルは、
システム上では一般ユーザー権限のファイルになります。
- コンテナ内で作成した一般ユーザー権限のファイルには、
システム上でユーザーID150000以降のユーザーIDが割り当てられます
- システム上で管理者権限が必要な操作は、コンテナ内であっても実施できません。
(管理者権限が必要なファイルの編集などが不可能)
- 各ユーザ毎にイメージが管理される分の、ストレージ容量を考慮する必要があります

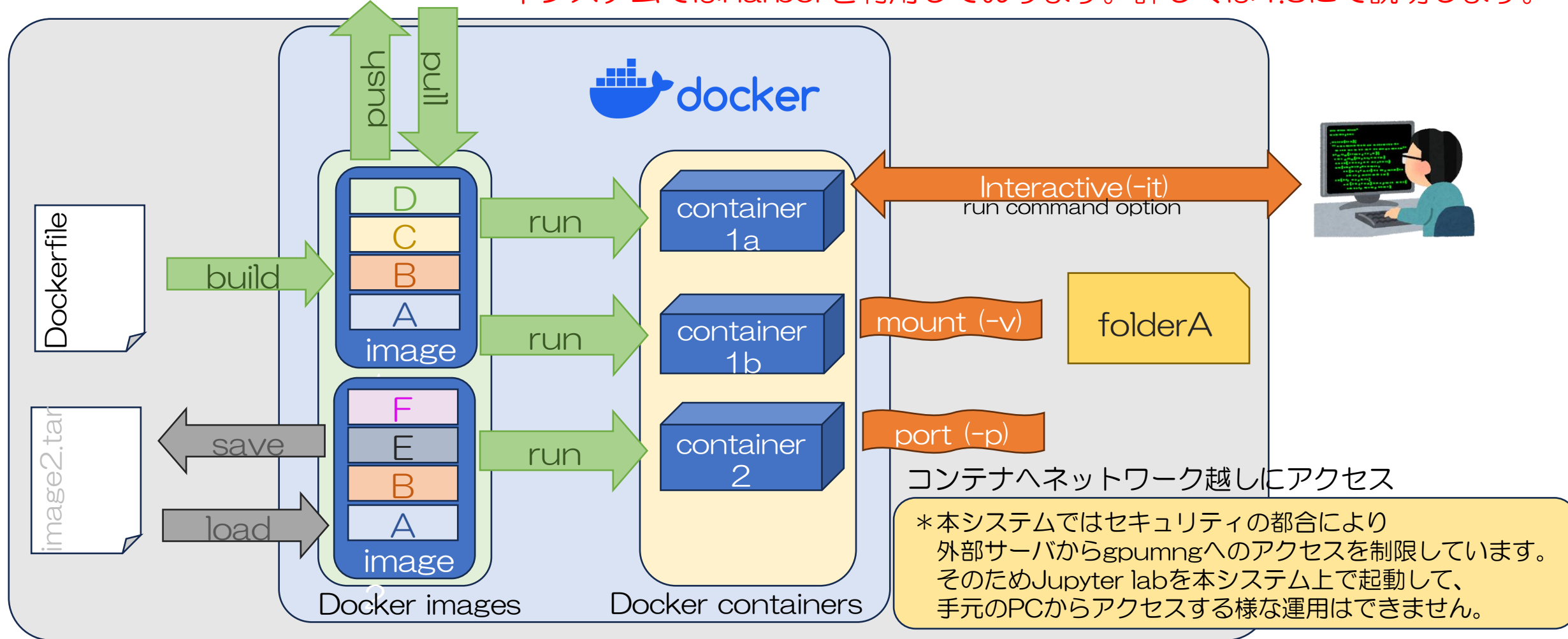
2-6 dockerコマンド概要図



Registry server

(Docker hub, NGC, Gitlab, docker registry, harbor, etc.)

*本システムではharborを利用しております。詳しくは4.3にて説明します。



2-7 docker Hub 注意事項

2026年8月現在のDocker Hubの利用規約は下記のとおりです

Docker Hubにアカウントを持っていないユーザー：

=> 同一IPアドレスのサーバーから6時間あたり100pull

Docker hubにアカウントを持っているユーザー：

=> アカウント1つにつき6時間あたり200pull

授業中に同時にdocker hubへアクセスをした際、
Docker hubへアクセスできなくなる可能性があります。

授業での利用の際は、Docker hubへの直接アクセスはできる限り避け、
Harborへあらかじめpushしたコンテナを利用してください。

Docker Hub usage and limits

 Ask Gordon  Copy Markdown  View Markdown

The following table provides an overview of the included usage and limits for each user type, subject to fair use:

User type	Pull rate limit per 6 hours	Number of public repositories	Number of private repositories
Business (authenticated)	Unlimited	Unlimited	Unlimited
Team (authenticated)	Unlimited	Unlimited	Unlimited
Pro (authenticated)	Unlimited	Unlimited	Unlimited
Personal (authenticated)	200	Unlimited	Up to 1
Unauthenticated users	100 per IPv4 address or IPv6 /64 subnet	Not applicable	Not applicable

規約は随時変更されるため、
最新の情報は下記URLから確認してください

<https://docs.docker.com/docker-hub/usage/>

▶ 3. Harborの利用方法



3 Harbor 利用方法

Harborは、ローカルなネットワーク環境内でDocker用レジストリサーバを立ち上げる際に利用されるアプリケーションで、下記機能を提供します

Harborの特徴：

- ①Docker からログインしてコンテナをpull/push
- ②ウェブブラウザからアクセスして、コンテナイメージの一覧を確認
- ③ウェブブラウザからアクセスして、コンテナイメージを削除



- 3-1 ウェブブラウザへのアクセス
- 3-2 利用可能なプロジェクト一覧
- 3-3 Harborへコンテナをpull/pushする

3-1 ウェブブラウザへのアクセス(1/2)

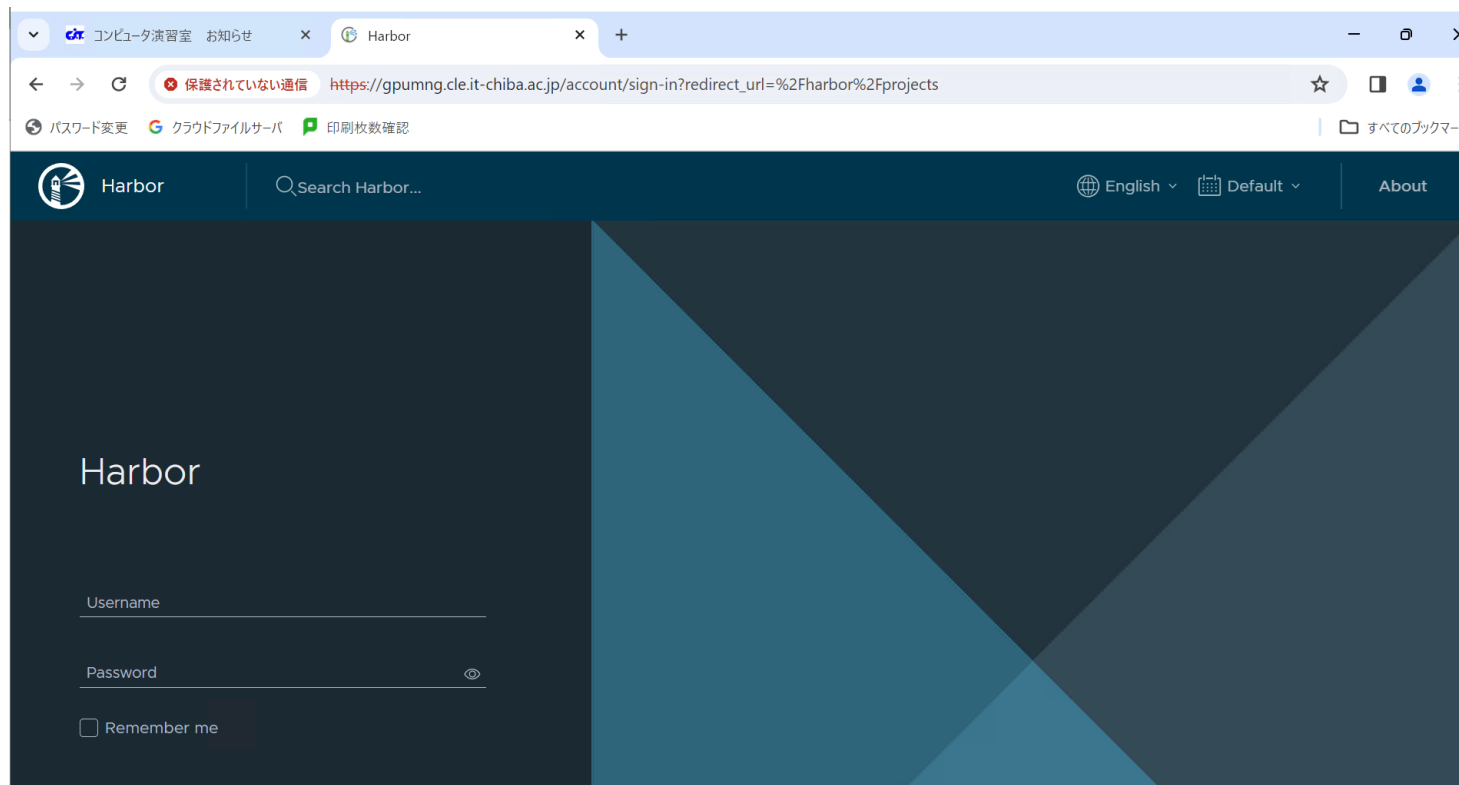
本システムのHarborには下記アドレスからアクセスしてください

新習志野

<https://gpumng.cle.it-chiba.ac.jp>

津田沼

<https://gpumng2.cle.it-chiba.ac.jp>



新習志野キャンパスと津田沼キャンパスのharbor/ファイルサーバはそれぞれ独立運用しており、データの同期はされない点にご注意ください

3-1 ウェブブラウザへのアクセス(2/2)

ログインすると、利用可能なプロジェクトが表示されます。
各プロジェクトをクリックすると、プロジェクト内に登録されているイメージが一覧で表示されます

Harbor

Search Harbor...

English Default S24B1999

Projects

Logs

Projects

Repositories

Public 2 Public 5

ACTION

All Projects

☐	Project Name	Access Level	Role	Type	Repositories Count	Creation Time
☐	class	Public	-	Project	0	2/27/24, 7:18 PM
☐	nvidia	Public	-	Project	5	2/27/24, 7:17 PM
☐	s24b1999	Private	Developer	Project	1	3/8/24, 3:26 PM

Page size 15 1 - 3 of 3 items

LIGHT

Harbor API V2.0

EVENT LOG

3-2 利用可能なプロジェクト一覧

本システムではコンテナイメージの移動・管理のためにHarborというregistryサーバを利用しています。各ユーザは以下3つのプロジェクトを利用することができます。

①nvidia

admin権限を有するユーザーのみpushやコンテナイメージの削除が可能なプロジェクトです。
権限を有しないユーザーは**コンテナのpullのみ**実行することができます。

用途：ユーザー全般が頻繁に利用するコンテナイメージを配置します。

②class

admin権限を有するユーザーおよび、teacherグループに所属しているユーザーがpull/push可能なプロジェクトです。
権限を有しないユーザーは**コンテナのpullのみ**実行することができます。

用途：授業用に先生が生徒へ配布するコンテナイメージを配置します。

③各ユーザーのプライベートプロジェクト

各ユーザー毎にプライベートなプロジェクトを作成しています。
権限を持たないユーザーには閲覧することができません。

容量は15GiBと設定しています（こちらは論理容量であり、実容量ではありません）。

用途：GPGPUサーバへコンテナイメージを移動させる際に利用してください。

3-3 Harborへコンテナをpull/pushする

利用したいコンテナイメージを、下記コマンドでpullしてください。
docker loginが事前に必要です。(詳しくは、1. gpumngへのログイン をご参照ください)

```
(gpumng)$ docker pull gpumng.cle.it-chiba.ac.jp/<プロジェクト名>/<イメージ名>:<タグ名>
```

例：nvidiaプロジェクト内のpytorchイメージのタグ名23.09-py3をpullする場合

```
(gpumng)$ docker pull gpumng.cle.it-chiba.ac.jp/nvidia/pytorch:23.09-py3
```

Harborへコンテナイメージをpushをする方法は、2-4をご参照ください

▶ 4. Slurm の利用方法



4 Slurm 利用方法

一般に共同利用計算環境では、ジョブ管理システムというものを導入し、サーバーリソース(CPU、GPU、メモリ等)を、各ユーザーが投入したジョブに適切に配分します。

ジョブ管理システムの特徴：

- ① 多人数が同時に利用した際にリソースの競合を防ぐ
- ② サーバーリソースを超えた場合、ジョブを待機状態にしておく
- ③ ユーザー毎に使用可能リソースなどを制限できる

本システムでは Slurm を導入しており、dockerコンテナもslurmから実行できるように設定しています



4-1 ジョブの投入: sbatch

4-2 dockerコンテナジョブの投入: sbatch

4-3 投入中のジョブの確認/キャンセル: squeue /scancel

4-4 ジョブ終了後の確認

4-1 ジョブの投入

Slurm からジョブを実行するには、下記の2ステップが必要です。

① バッチスクリプトを作成

submit_job.sh

```
#!/bin/bash
#SBATCH --partition=class
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=3
#SBATCH --gpus-per-task=1
#SBATCH --mem=90G

#SBATCH --mail-type=REQUEUE,END
#SBATCH --mail-user=<メールアドレス>
#~~~~~
#以下、実行したいコマンドを記述
#~~~~~
```

Slurm への設定は、冒頭に “#SBATCH” をつけてください

- class/researchパーティションを設定(5-2/5-3参照)
- 利用ノード数は 1 (本システムは固定)
- 並列数は 1 (本システムは固定)
- 使用するCPUコア数(3)を指定
- 使用するGPU枚数を(1)を指定
- 使用するメモリは(90GB)を指定

mail-type/userを指定することで
ジョブの実行・終了・強制終了などをメールに通知できます
(設定しなければメール通知されません)

mail-typeについて:
BEGIN,END,FAIL,REQUEUE,INVALID_DEPEND
等を複数設定できます。
全てのイベントを通知する場合はALLに設定してください。

② sbatch コマンドでバッチスクリプトを Slurm に投入

```
$ sbatch submit_job.sh
```

4-2 docker コンテナジョブの投入

Slurm から docker コンテナジョブを実行するには、前述のバッチスクリプトを
下のように変更してください

submit_docker_job.sh

実行したいコンテナを記入

```
#!/bin/bash
#SBATCH --partition=class
(中略)
#SBATCH --container=[コンテナ名]

#~~~~~
#以下、コンテナ内で実行したいコマンドを記述
#~~~~~
```

Slurm からコンテナを実行した際の注意事項：

1. rootアカウントとしてジョブが実行されます(ただし、システム上のrootとは異なります(2-5参照))
2. 各ユーザーのホームディレクトリが自動で/root以下にマウントされます
3. ログイン時のカレントディレクトリは、sbatchを実行したディレクトリです。
4. 共有データ置き場のファイルを/data以下に自動でマウントします(7-1参照)

4-3 投入中のジョブの確認/キャンセル

- slurm に投入したジョブの状態は、squeue コマンドで調べることができます。

```
$ squeue
```

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
21	class	TestJob	hpc	R	0:14	1	gpu01
22	class	TestJob	hpc	PD	0:00	1	(Resources)

status (ST)に関して：

R =>ジョブの実行中

PD =>リソースに空きがない等の理由で待機中

待機中の理由は(REASON)を参照してください。

特に、実行不可能なリソースを要求していた場合、(QOSMax***)と表示され、永遠に待機状態になる場合があります。

その際、下記コマンドでジョブのキャンセルを実行してください

- 投入中のジョブをキャンセルするには、scancel コマンドを使います。

※キャンセルに成功した場合、特にメッセージは出ません。上記 squeue コマンドの結果から消えたことを確認してください。

例: Job id 1001番目のジョブをキャンセル

```
$ scancel 1001
```

4-4 ジョブ終了後の確認

ジョブが終了すると、**ジョブを投入したディレクトリ**に下記のファイルが生成されます

- 標準出力+エラー出力結果ファイル

ファイル名 : slurm-<job id>.out

例 : slurm-1001.out

ジョブ実行時の**標準出力(stdout)**と**エラー出力(stderr)**の両方がこのファイルに記録されます。

※① 出力ファイル名を変えたい場合

ジョブ投入用バッチスクリプトに、

#SBATCH --output=<OutputFileName>-%j.out

という行を記載すると、ファイル名を指定できます。

(“%j” はジョブID)

※② エラー出力はファイルを分けたい場合

ジョブ投入用バッチスクリプトに、

#SBATCH --error=<ErrorFileName>-%j.out

という行を記載すると、ファイル名を指定しエラー出力のみのファイルを生成できます。

(“%j” はジョブID)

4-5 本システムで非推奨のジョブ

本システムでは、下記のようなコンテナ・ジョブの実行は非推奨となります。

- jupyter notebookなど、webブラウザからアクセスしてインタラクティブに操作を行うジョブ

slurmにはportを割り振る機能がありません。

そのため、他のユーザと利用ポートが競合する可能性があります。

また、演習中は各ジョブの実行時間に最大15分の制限があるため、本システム上で利用することは相性が悪いいため非推奨となります。

- ログインノード上でのコンテナの常駐

ログインノードに常駐プログラムを置くことは他ユーザーの使用リソースを圧迫するため非推奨となります。
また、ログインノードでは、**ログイン中でのみDockerが利用できる**ように設定しております

- コンテナ内でroot以外のユーザーになって動作するジョブ

rootless dockerでは、「**コンテナ外の一般ユーザー**」が「**コンテナ内のrootユーザー**」に対応するため、基本的にrootユーザー以外でジョブを実行する必要性がありません。

また、slurmからコンテナジョブを実行した場合、
コンテナ内でroot以外のユーザーを利用するとPermission Errorが発生したり、
コンテナ外のユーザーでは読み書きができないファイルが作成されるケースがあります。

4-6 演習モードへ移行した際の挙動

ジョブが実行・待機されている状態で演習時間に入った時、
実行されていたジョブは強制終了され、再び待機状態になります

```
hpc@gpumng:~/job$ hpc@gpumng:~/job$ squeue
JOBID PARTITION NAME USER ST TIME NODES NODELIST(REASON)
319 class .test jo hpc PD 0:00 1 (job queued in held state)
317 research .test_lo hpc PD 0:00 1 (job queued in held state)
hpc@gpumng:~/job$
```

出力ファイルからもjobがcancelされ、requeueされたことが確認できます。

```

===== ID ID Usage =====
| No running processes found |
+-----+
slurmstepd: error: *** STEP 319.0 ON gpu02 CANCELLED AT 2024-04-02T20:18:26 DUE TO JOB REQUEUE ***
srun: Job step aborted: Waiting up to 32 seconds for job step to finish.
slurmstepd: error: *** JOB 319 ON gpu02 CANCELLED AT 2024-04-02T20:18:26 DUE TO JOB REQUEUE ***
"slurm-319.out" 77L, 4578B

```

課外モードに再度突入後、requeue状態が解除され、ジョブを再度最初から実行します。
その際、出力ファイルは上書きされることに注意してください。

5. GPGPU計算機の利用にあたって



5. GPGPU計算機の利用にあたって

本システムは下記のような制限・設定をしております。
ご利用にあたって、下記仕様に注意して下さい。

- studentユーザーが使えるストレージ領域には制限があります(5-1で詳述)
- 演習時間中は他のユーザーはアクセスできません。
- 演習時間中と課外時間でslurmの**使用可能なリソース**が異なります(5-2/5-3で詳述)
- 演習時間中は、課外時間中に実行していたジョブを全て待機状態にします。
実行中だったジョブは強制終了した後、待機状態に再度設定されますので、
ジョブの途中経過を保存する等、各自で対策をお願いします。

5-1 ストレージ容量制限

studentユーザーが使えるストレージ領域は

新習志野 : 80GB

津田沼 : 240GB

です。

- 超過した場合、ログイン時に下記警告文が出ます。
警告文が出たら、適宜ファイルを削除してください

```
=====
Warning! File size is exceeded! please remove files!
=====
```

- 85/250GBを超過すると、新規の書き込みができなくなります。
- 現在の使用量を確認するには、下記コマンドを実行してください

```
(gpumng)$ hpcs_check_storage
```

5-2 Slurmのリソース制限(class)

演習中に利用することを目的としているclassパーティションは、
下記のようなリソース制限を行っています。

新習志野

津田沼

- 一人のユーザーが同時に投入できるジョブの最大数 : 1 job (実行中/待機中ジョブを含む)
- 1つのジョブで利用できる最大メモリ容量 : 90GB / 128GB
- 1つのジョブで利用できる最大CPUコア数 : 3core / 4core
- 1つのジョブで利用できる最大GPU数 : 1GPU
- 1つのジョブの最大実行時間 : 15分

*上記設定は運用中に管理者によって変更される可能性があります

5-3 Slurmのリソース制限(research)

演習時間外では、researchパーティションにもジョブを投入できます。
こちらは下記のようなリソース制限を行っています。

- 一人のユーザーが同時に投入できるジョブの最大数 : 2job (実行中/待機中ジョブを含む)
- 一人のユーザーが使用できる最大リソース : GPU10枚/CPU32コア/メモリ960GB
- 1つのジョブの最大実行時間 : 24時間
- 1つのジョブで使用できる最大リソース : GPU5枚/CPU16コア/メモリ480GB
* ノード跨ぎ不可

[2026年1月]利用者が増えたことをうけてリソースの変更を行いました

*上記設定は運用中に管理者によって変更される可能性があります

5-3 Slurmのリソース制限(research)

津田沼

演習時間外では、researchパーティションにもジョブを投入できます。
こちらは下記のようなリソース制限を行っています。

- 一人のユーザーが同時に投入できるジョブの最大数 : 2job (実行中/待機中ジョブを含む)
- 一人のユーザーが使用できる最大リソース : GPU2枚/CPU16コア/メモリ256GB
- 1つのジョブの最大実行時間 : 24時間
- 1つのジョブで使用できる最大リソース : GPU2枚/CPU16コア/メモリ256GB
* ノード跨ぎ不可

[2026年1月]利用者が増えたことをうけてリソースの変更を行いました

*上記設定は運用中に管理者によって変更される可能性があります

5-4 fairshare

[2026年8月] 利用者が増えたことをうけてfairshareを導入しました。

特定ユーザーによる独占状態を防ぐため、
直近で使用したユーザーのジョブの実行優先度が下がるfairshareを導入します。

Fairshareの仕組み：

- ①researchパーティションで実行する計算資源(CPU/GPUなど)の利用時間に応じてポイントが加算されます
- ②使用していない時ポイントは自動的に減っていきます。6時間でポイントが半減するように設定します。
- ③現在所持しているポイントが低いユーザーから優先的にジョブが実行されます
- ④1週間毎に加算したポイントをリセットします

Classパーティションでジョブを利用する場合、ポイントは加算されませんが、
Classパーティションで投入するジョブの優先度にはポイントが考慮されます

```
hpc@gpumng:~$ sshare -a
```

Account	User	RawShares	NormShares	RawUsage	EffectvUsage	FairShare
root				0	0.000000	
root	root	1		0	0.000000	0.000000
admin		1		0	0.000000	
admin	admin	1		0	0.000000	0.000000
gpustudents		1		0	0.000000	
gpustudents	A17C3008	1		0	0.000000	0.000000
gpustudents	A1931135	1		0	0.000000	0.000000
gpustudents	A20A1154	1		0	0.000000	0.000000
gpustudents	A2131066	1		0	0.000000	0.000000

現在のポイント状況は
sshareコマンドで確認できます

*上記設定は運用中に管理者によって変更される可能性があります

6. 開発事例



6 開発事例 1/3

- ① Dockerを利用できるBYOD端末上で、docker_build_testディレクトリとその中のファイルを作成

```
docker_build_test/  
└─ Dockerfile
```

Dockerfile

```
FROM nvcr.io/nvidia/pytorch:23.09-py3  
RUN apt-get update && apt-get install -y <packages>  
RUN pip install <packages>
```

← pip installやapt-getを実行して、
必要なパッケージをインストール

- ② docker_build_testディレクトリ内で下記を実行してコンテナイメージをビルド

```
(other)$ docker build -t gpumng.cle.it-chiba.ac.jp/<user名>/pytorch_test:1.0 .
```

- ③ Harborへコンテナイメージをpush

```
(other)$ docker push gpumng.cle.it-chiba.ac.jp/<user名>/pytorch_test:1.0 .
```

6 開発事例 2/3

- ④ コンテナ内で実行するpythonプログラムtest.pyを作成し、テストランを実行

```
(other)$ docker run --rm -v $(pwd):/root -w /root gpumng.cle.it-chiba.ac.jp/<user名>/pytorch_test:1.0 python test.py
```

Directoryのマウントとカレントディレクトリの変更

test.pyの実行

- ⑤ test.pyをgpumngにscpで移動

```
(other)$ scp test.py <user名>@gpumng.cle.it-chiba.ac.jp:test.py
```

- ⑥ gpumngに移動sshでログイン

```
(other)$ ssh <user名>@gpumng.cle.it-chiba.ac.jp
```

- ⑦ Harborからコンテナイメージをpull

```
(gpumng)$ docker pull gpumng.cle.it-chiba.ac.jp/<user名>/pytorch_test:1.0
```

6 開発事例 3/3

⑧ slurmに投入するためのバッチスクリプトを作成

```
(gpumng)$vim submit_test.job
```

```
#!/bin/bash
#SBATCH --partition=class
#SBATCH --nodes=1
#SBATCH --tasks-per-node=1
#SBATCH --cpus-per-task=3
#SBATCH --gpus-per-task=1
#SBATCH --mem=90G
#SBATCH --mail-type=REQUEUE,END
#SBATCH --mail-user=<メールアドレス>
#SBATCH --container=gpumng.cle.it-chiba.ac.jp/<user名>/pytorch_test:1.0

python test.py
```

⑨ slurmにジョブを投入

```
(gpumng)$sbatch submit_test.job
```

squeueを使ってジョブの実行状況を監視し、場合によってはscancelでジョブのキャンセルなどを実行してください

7. 教員向け情報

7-1. 共有データ置き場について

教員向け

機械学習で使用するデータセットなど、授業中に共有したいデータセットは
/home/local/data/share_data/以下に置いてください。

(学生ユーザーはこちらに置くことはできません。)

上記ディレクトリに置かれたデータは、

- 一定時間間隔で他計算サーバと同期をとります。

データは全て各サーバのローカルNVMe領域に保存されるため、

このディレクトリに置いたファイルへの参照はサーバー間通信が発生しません。

*リアルタイムでの同期は実行されません

即時反映させる場合は、下記コマンドを実行してください

教師ユーザも実行可能です。

```
sudo hpcs_sync_data
```

- slurmからコンテナジョブを実行した際、/dataとしてマウントされ、参照が可能です。
また、コンテナジョブから該当ディレクトリへの書き込みはできません。

7-2. 教員ユーザの権限について

教員向け

本システムでは、teacherグループに属しているユーザーは、管理者権限は付与していませんが、下記の操作が可能です。

- レジストリサーバのclassプロジェクトにコンテナイメージをpush
- /home/local/data/share_data以下にファイルを作成
- 下記コマンドにより、パスワードなしで学生ユーザー権限に移行可能

```
sudo -u <学生アカウント> -l
```

- hpcs_sync_dataはsudoをつけることで管理者として実行可能
- 下記コマンドを実行することで現在のサーバ設定が演習モードか確認することができます。

```
check_classmode
```

演習モード(test学科のみアクセス可能)

```
hpc@gpumng:~$ check_classmode
Classmode: on
Permitted Access Group: test
```

課外モード(誰でもアクセス可能)

```
hpc@gpumng:~$ check_classmode
Classmode: off
```

7-3. 授業での利用にあたって(1/2) 教員向け

本システムでコンテナを利用した授業を実施する際、下記の点に注意して運用してください

- 授業中に学生がDocker Hubへpullすることが頻繁に発生するような運用は非推奨となります

Docker Hubには同一IPアドレスからのpullに上限が定められており、
同じサーバーから1時間以内に10以上のpullを行うことができません。(Docker Hubアカウントがない場合)

授業前にHarborのclassプロジェクト上に授業用コンテナをpushし、harborから学生へ配布(pull)してください。

- ユーザーのホームディレクトリ上で読み書きが頻発するようなジョブはできる限り実行しないで下さい。

ユーザーのホームディレクトリはファイルサーバと計算サーバ間で常に同期をとっています。
読み書きが頻発した場合、サーバ間のデータの同期をとる処理が頻発し、
システム全体のパフォーマンスが低下します。

授業で利用する予定のデータセット等、学生には読み込み権限のみ付与すればよいファイルは、
事前に/home/local/data/share_data/配下に配置してください

こちらに配置したデータは、一定時間間隔で各計算サーバに搭載されているNVMeストレージへコピーされます。
こちらのストレージに置かれたデータであれば、計算中にファイルサーバーへのデータの読み書きが発生しません。

7-3. 授業での利用にあたって(2/2)

教員向け

本システムでコンテナを利用した授業を実施する際、下記の点に注意して運用してください

- コンテナネットワークは下記の仕様になっています。

①外部から本システム上で稼働しているコンテナへのアクセスは不可(jupyter lab等)
セキュリティ上の理由により、ポートを閉じています。

②クラスターシステム内のコンテナ間アクセス ⇒ 可
(ただし、ポート競合にはご注意ください)

パフォーマンス上の理由により、slurmから起動するコンテナは
デフォルトでホストネットワークに設定しています。
そのため、すべてのコンテナ間で、ホストネットワークを使った通信が行われます。
同じポートを利用した場合などで、ポート競合によってサーバーが立ち上がらない、
といった問題が起きる可能性がありますのでご注意ください。